SISTEMAS INTELIGENTES (PET1702)

Projeto 4 - Classificação Automática de Sinais de Radar Ionosférico usando uma Rede Adaline

Componentes: **JEFFERSON BEZERRA DA GAMA (2025022550)**
LAVÍNIA DANTAS DE MESQUITA (2025022536)

I. OBJETIVO

O presente projeto tem como objetivo implementar e avaliar uma rede Adaline (Adaptive Linear Neuron) para a classificação de sinais de radar ionosférico, utilizando o dataset Ionosphere.

Busca-se analisar o impacto da taxa de aprendizagem no processo de convergência do modelo, bem como avaliar seu desempenho por meio de métricas de classificação, como matriz de confusão, acurácia, precisão, recall e F1-score.

Para o desenvolvimento do modelo, é utilizado o dataset Ionosphere, coletado pelo sistema de radar Goose Bay e disponibilizado pelo UCI Machine Learning Repository. O conjunto de dados é composto por 351 amostras descritas por 34 atributos contínuos, onde o rótulo binário classifica o retorno como good (presença de estrutura) ou bad (ausência de estrutura coerente).

II. PARTE TEÓRICA

A Adaline (Adaptive Linear Neuron) é um modelo de rede neural artificial baseado em regressão linear, que utiliza a regra Delta para ajuste dos pesos por meio do gradiente descendente.

Diferentemente do Perceptron, Adaline realiza a atualização dos pesos com base no erro contínuo, utilizando a função de custo baseada no erro quadrático médio (MSE), o que permite uma análise mais estável do processo de aprendizado.

A atualização dos pesos é dada por: $w = w + \eta * x * erro$, onde η representa a taxa de aprendizagem, x são as entradas e erro é a diferença entre o valor real e o valor previsto.

A escolha adequada da taxa de aprendizagem é fundamental: valores muito altos podem causar instabilidade e divergência, enquanto valores muito baixos podem resultar em convergência lenta. Para avaliação do modelo, são utilizadas métricas como acurácia, precisão, recall e F1-score, além da análise da matriz de confusão.

III. PARTE PRÁTICA

Inicialmente, foi utilizado o dataset Ionosphere, contendo 351 amostras com 34 atributos contínuos relacionados a sinais de radar. A variável alvo foi recodificada para valores bipolares, sendo +1 para a classe "good" e -1 para a classe "bad". Na etapa de pré-processamento, foram removidos atributos constantes e realizada a normalização dos dados utilizando o Z-score, garantindo média zero e variância unitária para todas as variáveis.

```
# Recodificando os rótulos: 'g' para +1 e 'b' para -1
y = np.where(y_raw == 'g', 1, -1)
# Removendo atributos constantes (colunas com apenas 1 valor único)
X_raw = X_raw.loc[:, X_raw.nunique() > 1]
```

Fonte: Autoria Própria

Os dados foram divididos em conjuntos de treino (80%) e teste (20%), utilizando estratificação para preservar a proporção das classes.

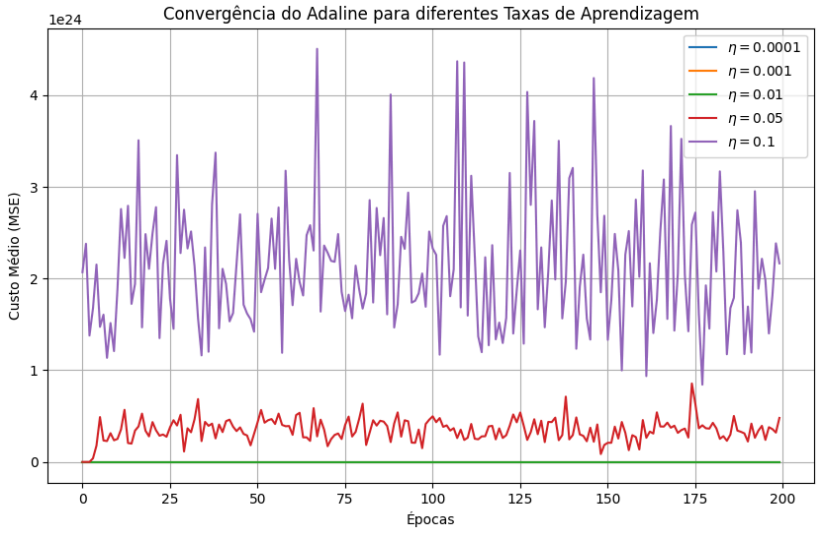
```
# Divisão em treino (80%) e teste (20%) com amostragem estratificada
X_train, X_test, y_train, y_test = train_test_split(
    X_raw, y, test_size=0.2, random_state=42, stratify=y
)
```

Fonte: Autoria Própria

A normalização dos dados foi essencial para garantir a estabilidade do treinamento da rede Adaline. Como o algoritmo utiliza a regra Delta baseada em gradiente descendente, diferenças de escala entre os atributos poderiam

causar atualizações desproporcionais dos pesos, levando à instabilidade. Já a padronização via Z-score garantiu que todas as variáveis contribuíssem de forma equilibrada para o processo de aprendizado, resultando em maior estabilidade e melhor convergência.

Em seguida, foi implementado o modelo Adaline e realizado um estudo com diferentes taxas de aprendizagem (η), mantendo fixo o número máximo de épocas (200). Para cada valor de η, foi analisada a curva de convergência da função de custo.



Fonte: Autoria Própria

A análise das curvas de convergência mostrou que taxas de aprendizagem elevadas, como η = 0.1, causam instabilidade no treinamento, com valores de erro elevados e comportamento oscilatório, indicando divergência. Por outro lado, taxas muito baixas, como η = 0.0001, resultam em aprendizado lento, com pouca redução do erro ao longo das épocas.

A taxa η = 0.01 apresentou o melhor desempenho, com convergência estável e redução consistente da função de custo, sendo selecionada como a melhor configuração para o modelo. Com base nessa análise, foi selecionada a melhor taxa de aprendizagem para treinamento final do modelo e posterior avaliação no conjunto de testes.

```
# ── 3. Classificação e métricas com o melhor eta ──
# Substituindo pelo melhor valor identificado no gráfico!
melhor_eta = 0.01

adaline_final = SGDClassifier(
    loss='squared_error',
    learning_rate='constant',
    eta0=melhor_eta,
    max_iter=200,
    shuffle=True,
    random_state=42
)

# Treinamento com o melhor eta
adaline_final.fit(X_train, y_train)

# Previsão nas amostras de teste
y_pred = adaline_final.predict(X_test)

# Cálculo das métricas
```

```python
conf_matrix = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred) # Precisão para a classe +1
rec = recall_score(y_test, y_pred)     # Recall para a classe +1
f1 = f1_score(y_test, y_pred)          # F1-Score para a classe +1

print("--- Avaliação do Modelo ---")
print(f"Melhor Taxa de Aprendizagem escolhida: {melhor_eta}")
print("\nMatriz de Confusão:")
print(conf_matrix)
print(f"\nAcurácia Global: {acc:.4f}")
print(f"Precisão (+1)  : {prec:.4f}")
print(f"Recall (+1)    : {rec:.4f}")
print(f"F1-Score (+1)  : {f1:.4f}")

# Métricas detalhadas para a classe bad (-1)
prec_bad = precision_score(y_test, y_pred, pos_label=-1)
rec_bad = recall_score(y_test, y_pred, pos_label=-1)

print("\nMétricas para a classe 'bad' (-1):")
print(f"Precisão (-1)  : {prec_bad:.4f}")
print(f"Recall (-1)    : {rec_bad:.4f}")
```

Fonte: Autoria Própria

```
--- Avaliação do Modelo ---
Melhor Taxa de Aprendizagem escolhida: 0.01

Matriz de Confusão:
[[23  2]
 [ 0 46]]

Acurácia Global: 0.9718
Precisão (+1)  : 0.9583
Recall (+1)    : 1.0000
F1-Score (+1)  : 0.9787

Métricas para a classe 'bad' (-1):
Precisão (-1)  : 1.0000
Recall (-1)    : 0.9200
```

Fonte: Autoria Própria

O modelo apresentou um excelente desempenho na classificação das amostras de teste. A matriz de confusão indicou poucos erros, com 23 classificações corretas para a classe "bad" e 46 para a classe "good".

A acurácia global foi de aproximadamente 97%, evidenciando alta taxa de acertos. A precisão para a classe positiva (+1) foi de 0.96, enquanto o recall atingiu o valor máximo (1.0), indicando que todas as amostras dessa classe

foram corretamente identificadas. Para a classe "bad" (-1), o modelo apresentou precisão de 1.0 e recall de 0.92, indicando excelente desempenho, com pequena taxa de erros.

De forma geral, o modelo demonstrou alta capacidade de generalização e separação entre as classes, porém o erro mais crítico ocorre quando uma amostra da classe "bad" é classificada como "good" (falso positivo), pois isso levaria o sistema a interpretar incorretamente que a ionosfera está em condições adequadas. Esse tipo de erro pode comprometer aplicações críticas, como telecomunicações e sistemas de navegação, tornando essencial sua minimização para garantir maior confiabilidade do sistema.

IV. CONCLUSÕES

A partir dos resultados obtidos, conclui-se que a rede Adaline é uma abordagem eficaz para a classificação de sinais ionosféricos, apresentando alto desempenho e capacidade de generalização. A análise das taxas de aprendizagem evidenciou a importância da escolha adequada desse parâmetro para garantir estabilidade e eficiência no treinamento.

O modelo alcançou excelente desempenho, com alta acurácia e métricas consistentes, demonstrando sua capacidade de separar corretamente as classes, no entanto, observou-se a importância de considerar o impacto dos erros de classificação, especialmente aqueles relacionados à classe "bad", que podem comprometer aplicações reais.

Como trabalhos futuros, sugere-se explorar técnicas adicionais de regularização, bem como comparar o desempenho da Adaline com outros modelos mais complexos, como redes neurais multicamadas.

REFERÊNCIAS

UCI Machine Learning Repository. **Ionosphere Dataset.** Disponível em: <https://archive.ics.uci.edu/ml/datasets/Ionosphere>. Acesso em: 27 abr. 2026.

SCIKIT-LEARN. Linear Models - **SGDClassifier.** Disponível em: <https://scikit-learn.org/stable/modules/sgd.html>. Acesso em: 27 abr. 2026.

SOUZA, Pedro Thiago Valério de. **Intelligent Systems - Slides.** GitHub, 2026. Disponível em: <https://github.com/pedrothiag/intelligent-systems/tree/main/slides>. Acesso em: 27 abr. 2026.